Ridge and Lasso Regression - Regularization Methods with Mathematical Proofs – with Demo simulation

Author: Jean de Dieu UWINTWALI
UNIVERSITY OF RWANDA
Department: Computer and Software Engineering
Year: 4
Email: uwintwalijeandedieu3@gmail.com

Mathematical Analysis and How Regularization Works Behind the Scenes in Data Science Context
Focus area: L1/L2 Penalties, Sparsity, Sub-gradient analysis, Feature Selection

Github: https://github.com/UWINTWALI/ridge-lasso-regularization.git
Live Demo: https://feature-selection-regularization-technique.streamlit.app/

Contents

Introduction to Regularization 3
Linear Regression: The Starting Point 3
The Linear Model 3
Ordinary Least Squares (OLS) 3
The OLS Objective Function 3
Ridge Regression: Mathematical Foundation 4
Ridge Objective Function 4

Ridge Solution: Derivation 4
Why Ridge Works: The Stabilization Effect 5
Example: Ridge with Correlated Features 5
Lasso Regression: The Mathematics of Sparsity 6
Lasso Objective Function 6
Why Lasso Creates Exact Zeros: The Mathematical Proof 6
Understanding Λ: Why Different Values for Ridge vs Lasso 9
Optimization: How Algorithms Find Ridge and Lasso Solutions 10
Ridge: Gradient Descent - Simple 10
Lasso: Subgradient 10
Ridge vs Lasso: Summary Table with Λ Guidance 11
Selecting Optimal Λ via Cross-Validation 11
Conclusion: Mathematics Behind Regularization 12

1. Introduction to Regularization

Linear regression is fundamental, but standard OLS (Ordinary Least Squares) fails when you have many features relative to samples, multicollinearity, or high-imensional data. The solution is regularization that adds a penalty term that prevents the model from learning noise. This document goes beyond intuition. It shows the mathematics behind Ridge and Lasso, prove why Lasso creates exact zeros or feature selection, explain why different penalty strengths are needed, and demonstrate everything with real numerical examples.

Understanding this mathematics is crucial for data scientists since it reveals what your regularization is actually doing at the optimization level, allowing you to make informed decisions about regularization strength and model architecture.

2. Linear Regression: The Starting Point

2.1. The Linear Model

We want to predict a target variable y using features or predictors, X. We assume a linear relationship:

$y = X \cdot \beta + \text{epsilon}$

where:

y = target values we want to we predict

X = feature matrix as input data

β = coefficients

epsilon = errors/noise

For example, when predicting house price, the function will be like as follow.

y = 150,000 + 300 × size + 50,000 × bedrooms + 10,000 × age + ….

In this case here, the coefficients (300, 50,000, 10,000) tell us that each extra square foot adds $300 to price, each bedroom adds $50,000, and so on…

2.2. Ordinary Least Squares (OLS)

2.2.1. The OLS Objective Function

OLS finds coefficients by minimizing the sum of squared prediction errors.

$L_{OLS}(\beta) = \sum_{i=1}^{n}(y_i - x_i \cdot \beta)^2$

In matrix form, it becomes: $L_{OLS}(\beta) = ||y - X \cdot \beta||_2^2$

Solution: $\beta_{OLS} = (X' \cdot X)^{-1} \cdot X' \cdot y$

The issue with this OLS is that when features are correlated (multicollinearity), the matrix X'X becomes nearly singular. This makes $(X'X)^{-1}$ unstable, producing enormous, unreliable coefficients.

2.3. Ridge Regression: Mathematical Foundation

2.3.1. Ridge Objective Function

Ridge adds an L2 penalty (the sum of squared coefficients) to the OLS objective:

$L_{Ridge}(\beta) = ||y - X * \beta||_2^2 + \lambda . ||\beta||_2^2$

$L_{Ridge}(\beta) = \sum_{i=1}^{n}(y_i - x_i . \beta)^2 + \lambda . \sum_{j=1}^{p} \beta_j^2$ = [OLS error] + [penalty for large coefficients]

Where:

$\lambda >= 0$ is the regularization parameter

λ (λ) = strength of penalty

- $\lambda = 0$: Pure OLS with no penalty
- $\lambda = 0.1$: Weak penalty
- $\lambda = 1.0$: Strong penalty
- $\lambda = 10$: Very strong penalty

Interpretation:

Ridge regression balances two goals: fitting the data well and keeping the model coefficients small. The first term minimizes the prediction error, while the second term penalizes large coefficient values. The regularization parameter λ controls this trade-off. When λ=0, the model becomes ordinary least squares (OLS). As λ increases, the coefficients shrink toward zero, which helps reduce overfitting and improves the model's ability to generalize to new data.

2.3.2. Ridge Solution: Derivation

To obtain the Ridge regression estimator, we minimize the Ridge loss function with respect to the coefficient vector β. This is done by computing the gradient of the loss function and setting it equal to zero to find the optimal solution.

STEP #1: Compute gradient

$\frac{\partial LRidge}{\partial \beta} = -2 . X' . (y - X . \beta) + 2 . \lambda . \beta = 0$

STEP #2: Simplify

$-2.X'.y + 2.X'.X.\beta + 2.\lambda.\beta = 0$

$X'.X.\beta + \lambda.\beta = X'.y$

$(X'.X + \lambda.I).\beta = X'.y$

STEP #3: Solve for β

$\beta_{Ridge}(\lambda) = (X'.X + \lambda.I)^{-1} . X' . y$

2.3.3. Why Ridge Works: The Stabilization Effect

Mathematical Insight: By adding λ.I to the diagonal of X'X, Ridge makes the matrix well-conditioned, which is far from singular. The larger the λ is, the more stable the inversion.

When $\lambda = 0$: β_{Ridge} reduces to OLS (unstable with correlated features)

When $\lambda > 0$: The λ.I term pushes eigen values away from zero, stabilizing the inversion

When λ very large: β_{Ridge} approaches zero, extreme regularization.

Example: Ridge with Correlated Features

PROBLEM: Suppose we want to predict house prices using two explanatory variables: house size (ranging from 200–500 square feet) and number of bedrooms (ranging from 2–5). In practice, these two features are often highly correlated, since larger houses tend to have more bedrooms. This correlation introduces multicollinearity in the regression model, which can make the ordinary least squares (OLS) estimates unstable. Ridge regression is used to address this issue by stabilizing the coefficient estimates through regularization.

```
DATA:
y = [100, 150, 200, 250] # As prices in thousands
X =     [[200, 3],
        [250, 3],
        [350, 4],
        [450, 5]]
STEP #1: Compute X'.X (shows the correlation problem)
X'.X = [[650000, 8450],
        [8450, 119]]
Condition number is very large (nearly singular!)
STEP #2: Try OLS (λ = 0)
βOLS = (X'.X)^-1 . X' . y
X'.X is nearly singular. Inversion fails or is unstable
Result: βOLS = [0.8, -15.2]  #huge, contradictory values!
Interpretation: Size coefficient = 0.8 (reasonable)
Bedrooms = -15.2 # negative?! More bedrooms DECREASE price?
This contradicts reality and it is a sign of overfitting or multicollinearity
STEP #3: Apply Ridge with λ = 10
X'.X + λ.I = [[650010, 8450],
              [8450, 129]]
Compute: βRidge(10) = (X'.X + 10.I)^-1 . X' . y
Result: βRidge = [0.45, 2.1] # With λ of 10 we can get small, sensible values
Interpretation: Size = 0.45 (It is reasonable)
```

Bedrooms = 2.1 (positive, and makes sense!)
STEP #4: Try Ridge with λ = 100 (stronger penalty)
$\beta_{Ridge}(100)$ = [0.20, 0.8] (even smaller values)
COMPARISON:
OLS (λ=0): [0.8, -15.2] : unreliable
Ridge (λ=10): [0.45, 2.1] : stable
Ridge (λ=100): [0.20, 0.8] : very stable but underfitting

KEY OBSERVATION:

- λ = 0: Unstable due to multicollinearity
- λ = 10: Good balance (stabilizes without over-penalizing)
- λ = 100: Over-penalized (loses information)
- Optimal λ can be found via cross-validation

NOTICE: Ridge doesn't eliminate multicollinearity, but it handles it by stabilizing the inversion. Different λ values produce different stability levels. Too small (λ=0) remains unstable; too large (λ=100) overfits. The optimal λ can be found through Cross-Validation.
WHY λ=10 for Ridge in our case: This value stabilizes X'X without over-penalizing the coefficients. If we used λ=0.1, it wouldn't help much. If we used λ=1000, all coefficients would shrink to near-zero, losing predictive power.

2.4. Lasso Regression: The Mathematics of Sparsity

2.4.1. Lasso Objective Function

While Ridge regression shrinks coefficients toward zero, it rarely makes them exactly zero. In contrast, Lasso regression introduces a different type of regularization that can force some coefficients to become exactly zero, effectively performing automatic feature selection. This property makes Lasso particularly useful when dealing with high-dimensional datasets where only a subset of predictors is truly relevant.

$L_{Lasso}(\beta) = ||y - X.\beta||_2^2 + \lambda . ||\beta||_1$
$L_{Lasso}(\beta) = \sum_{i=1}^{n}(y_i - x_i.\beta)^2 + \lambda . \sum_{j=1}^{p}|\beta_j|$
Key difference from Ridge: $\sum(|\beta_j|)$ instead of $\sum(\beta_j^2)$

2.4.2. Why Lasso Creates Exact Zeros: The Mathematical Proof

This is the key insight that separates Ridge from Lasso. We prove why the absolute value
penalty forces coefficients to EXACTLY zero while squared penalties only shrink them.

PROOF #1: Understanding the Penalty Function

Ridge penalty: $f_{Ridge}(\beta) = \beta^2$

Derivative: $\frac{\partial f_{Ridge}}{\partial \beta} = 2.\beta$

At $\beta = 0$: $\frac{\partial f_{Ridge}}{\partial \beta} = 0$

Conclusion: The gradient is smooth, never hits zero unless β already is zero

Lasso penalty: $f_{Lasso}(\beta) = |\beta|$

Derivative: $\frac{\partial f_{Lasso}}{\partial \beta} = \text{sign}(\beta)$

At $\beta = 0$: NOT DIFFERENTIABLE!

Sub-gradient at 0: any value in [-1, 1]

NOTICE:

The absolute value has a CORNER at zero. This corner is special: it allows coefficients to "stick" at zero during optimization.

PROOF #2: When Does Lasso Force a Coefficient to Zero?

At the optimal solution, the sub-gradient condition is: $-2.X_j'.(y - X.\beta) + \lambda.\text{sign}(\beta_j) = 0$

where X_j is the jth column of X

It can be rearranged as follow:

$2.X_j'.(y - X.\beta) = \lambda.\text{sign}(\beta_j)$

Now consider when $\beta_j = 0$ (coefficient becomes zero):

$2.X_j'.y = \lambda.\text{sign}(0)$

But sign(0) is NOT a single value; it's the interval [-1, 1]

So: $2.X_j'.y$ must satisfy: $-\lambda <= 2.X_j'.y <= \lambda$

Or equivalently:

$|2.X_j'.y| <= \lambda$

RESULT:

If $|2.X_j'.y| <= \lambda$, then $\beta_j = 0$ is OPTIMAL

If $|2.X_j'.y| > \lambda$, then β_j must be non-zero

This threshold condition is what forces exact zeros! Features with small correlation to residuals get eliminated.

MATHEMATICAL INSIGHT: The absolute value penalty $|\beta|$ creates a "threshold effect". If the gradient of the OLS loss is small enough (smaller than λ), Lasso can keep the coefficient at exactly zero. Ridge's squared penalty never has this threshold, all coefficients are pushed slightly but none reach exactly zero.

Example:
We aim to predict customer loyalty using 4 features: two relevant (years as customer, purchase frequency) and two irrelevant (age, store location). OLS regression assigns non-zero coefficients to all features, including noise. Ridge regression shrinks coefficients but never eliminates irrelevant ones. Applying Lasso with $\lambda = 0.3$, we compute the gradient of the loss for each feature and compare it to the threshold λ. Features 1 and 2 have large gradients, so their coefficients remain non-zero, while features 3 and 4 have small gradients, satisfying $|2\, X_j^T \text{residual}| \leq \lambda$, and are set exactly to zero. The resulting Lasso model retains only the relevant features, demonstrating automatic feature selection based on mathematical optimization, unlike Ridge or OLS.

DATA:
y = [8, 7, 9, 8, 7] (loyalty scores 0-10)
X = [[2, 5, 30, 1],
[1, 3, 25, 2],
[3, 8, 35, 1],
[2, 6, 28, 3],
[1, 4, 22, 2]]

STEP #1: Train OLS. No regularization
β_{OLS} = [0.8, 0.6, 0.05, -0.02]. All 4 coefficients are non-zero including noise

STEP #2: Reason Ridge doesn't eliminate irrelevant features
Ridge with $\lambda = 0.1$, It provides smaller coefficients as follow. Still all non-zero, just smaller. Ridge shrinks coefficients but never eliminates them.

β_{Ridge} = [0.75, 0.55, 0.04, -0.015]

STEP #3: Apply Lasso with $\lambda = 0.3$
Calculate gradient of OLS loss for each feature:
grad_1 = $2.X_1'.(y - X.\beta)$ = 1.2 (large)
grad_2 = $2.X_2'.(y - X.\beta)$ = 0.9 (large)
grad_3 = $2.X_3'.(y - X.\beta)$ = 0.08 (small)
grad_4 = $2.X_4'.(y - X.\beta)$ = 0.15 (small)

Apply the Lasso threshold condition:
$|\text{grad}_j| <= \lambda$?
Feature 1: |1.2| > 0.3? YES, β_1 stays non-zero
Feature 2: |0.9| > 0.3? YES, β_2 stays non-zero
Feature 3: |0.08| > 0.3? NO, $\beta_3 = 0$ (ELIMINATED)

Feature 4: $|0.15| > 0.3$? NO, $\beta_4 = 0$ (ELIMINATED)

FINAL RESULT of Lasso optimization:
$\beta_{Lasso} = [0.8, 0.6, 0, 0]$

COMPARISON:
OLS: [0.8, 0.6, 0.05, -0.02]
Bring back all the features' coefficients and including noise.
Ridge (λ=0.1): [0.75, 0.55, 0.04, -0.015]
Bring back all the features' coefficients but shrinks them and became smaller.
Lasso (λ=0.3): [0.8, 0.6, 0, 0]
Lasso eliminate irrelevant feature by setting their coefficients to zero.

INSIGHT: The Lasso threshold condition $|2 . X_j' . residual| <= \lambda$ determines which features survive. Features with small gradient (weak correlation to residuals) are eliminated. This is automatic feature selection based on mathematical optimization.

WHY λ=0.3 for Lasso (different from Ridge's λ=10):

- Ridge λ=10: Stabilizes the matrix X'X and addresses multicollinearity
- Lasso λ=0.3: Controls feature elimination using threshold. This is also interpreted as feature selection.

These serve different purposes, so different values are optimal. Lasso uses much smaller λ because the threshold is directly on gradient magnitudes (typically 0.1-1), while Ridge operates on the full condition number of X'X, typically 1-100.

2.5. Understanding Λ: Why Different Values for Ridge vs Lasso

Ridge Λ =10, for instance:
Ridge operates on eigenvalues of X'X. When features are correlated, X'X has small eigenvalues (near-singularity). The condition number is large (ratio of largest to smallest
eigenvalue). Adding **λ . I** increases all eigenvalues equally. To stabilize effectively, λ must be comparable to the smallest eigenvalues, typically 0.1 to 100.
Lasso Λ = 0.3, for instance:
Lasso operates on correlation between features and residuals. The optimal λ depends on the magnitude of $2 . X' . (y - X . \beta)$. This is typically much smaller (0.01 to 10) because normalized features and residuals are usually in [-1, 1] range, making 2 . X' . residual typically 0.1 to 5.

General Rules for Cross-Validation:
Ridge: Test λ values like [0.1, 1, 10, 100, 1000]
Lasso: Test λ values like [0.001, 0.01, 0.1, 1, 10]
Different scales are needed because they operate on different scales of the optimization problem.

2.6. Optimization: How Algorithms Find Ridge and Lasso Solutions

2.6.1. Ridge: Gradient Descent - Simple

Ridge loss: $L_{Ridge}(\beta) = ||y - X . \beta||^2 + \lambda . ||\beta||^2$
Gradient: $\frac{\partial f_{Ridge}}{\partial \beta} = -2 . X' . (y - X . \beta) + 2 . \lambda . \beta$

Update rule (gradient descent):
$\beta_{new} = \beta_{old}$ - step_size . gradient

The gradient is SMOOTH everywhere, so standard gradient descent works.
Algorithm starts at random β, repeatedly move in negative gradient direction
Converges to: $\beta_{Ridge} = (X' . X + \lambda . I)^{-1} . X' . y$

2.6.2. Lasso: Subgradient

Lasso loss: $L_{Lasso}(\beta) = ||y - X . \beta||^2 + \lambda . ||\beta||_1$
When $|\beta|$ is NOT differentiable at $\beta = 0$, use SUBGRADIENT instead of gradient
For any β_j:
If $\beta_j > 0$: subgradient = $-2 . X_j' . (y - X . \beta) + \lambda$
If $\beta_j < 0$: subgradient = $-2 . X_j' . (y - X . \beta) - \lambda$
If $\beta_j = 0$: subgradient $\in [-2 . X_j' . (y - X . \beta) - \lambda, -2 . X_j' . (y - X . \beta) + \lambda]$

Optimality condition:
β_j is optimal at 0 if 0 is in the subgradient interval i.e if $|2 . X_j' . y| <= \lambda$
Algorithm: Coordinate descent

- ✓ Update one β_j at a time
- ✓ Apply soft-thresholding: if $|\text{gradient}| < \lambda$, set $\beta_j = 0$

NOTICE: Lasso's subgradient has a special property at zero: it's an INTERVAL, not a single value. This interval allows zero to be optimal even when the gradient is small but non-zero. Ridge's smooth gradient never has this property. This is why Lasso creates exact zeros while Ridge only shrinks.

Ridge vs Lasso: Summary Table with Λ Guidance

Aspect	**Ridge(L2)**	**Lasso(L1)**
Penalty	$\lambda \cdot \sum(\beta^2)$	$\lambda \cdot \sum(\lvert\beta\rvert)$
Gradient	Smooth: $2 \cdot \beta$	Non-smooth: sign(β)
Closed-Form	Yes: $(X'X+ \lambda \cdot I)^{-1} \cdot X' \cdot y$	No: iterative algorithms
Coefficients	Shrink smoothly toward zero	Jump to exactly zero
Feature Selection	No: all features stay	Yes: removes irrelevant
Typical Lambda(λ) Range	0.1 to 1000	0.001 to 10
Lambda(λ) Selection	Via cross-validation	Via cross-validation
Best For	Many small important effects	Few large important effects
Multicollinearity	Handles well	Handles well but selects one
Interpretability	All features present	Simple: only selected features

3. Selecting Optimal Λ via Cross-Validation

Cross-validation finds the λ that minimizes prediction error on unseen data.

METHOD: Cross-Validation
Split data into k folds (For example, folds = 5)

STEP 1: Divide data into 5 folds (5 pieces)
STEP 2: For each lambda value (try: 0.001, 0.01, 0.1, 1, 10):

- Train on folds 1 to 4, and test on fold 5
- Record test error
- Train on folds 1,3 to 5, and test on fold 2
- Record test error
- Repeat for all 5 combinations
- Average the 5 test errors

STEP 3: Choose lambda with lowest average test error

WHY THIS WORKS:
Test error on unseen data tells us which lambda generalizes best.
We're not mislead by training error, which always decreases with smaller lambda.

RECOMMENDATION: Always use cross-validation (or hold-out test set) to choose lambda. Never choose based on training error alone.

Ridge Example Grid: $\lambda \in [0.1, 1, 10, 100]$

Lasso Example Grid: $\lambda \in [0.001, 0.01, 0.1, 1]$

The selected λ provides a trade-off between accurately fitting the training data (when λ is small) and controlling model complexity by shrinking coefficients keeping them stable in Ridge or sparse in Lasso (when λ is large).

4. Conclusion: Mathematics Behind Regularization

1. Ridge stabilizes by adding $\boldsymbol{\lambda \cdot I}$ to X'X, improving the condition number. The closed-form solution $\beta_{Ridge} = (X'X + \lambda \cdot I)^{-1} \cdot X' \cdot y$ is always computable.
2. Lasso creates exact zeros because the absolute value penalty $|\beta|$ has a corner at zero. The subgradient at zero is an interval, allowing optimization to "stick" at zero when $|2 \cdot X_j' \cdot \text{residual}| <= \lambda$.
3. Different λ scales matter: Ridge typically needs $\lambda \in [0.1, 1000]$ because it operates on eigenvalues of X'X. Lasso typically needs $\lambda \in [0.001, 10]$ because it operates on correlation magnitudes, much smaller scale.
4. Feature selection in Lasso is automatic: Features with gradient magnitude below λ are eliminated by the threshold condition. This makes Lasso ideal for high-dimensional data where most features are irrelevant.

For Data Scientists:

Understanding these mathematical mechanics transforms how you use regularization. You'll know why to choose different λ scales, why Lasso automatically selects features, why
Ridge stabilizes multicollinearity, and how to interpret the results with confidence

Practical Impact: Regularization transforms overfitting models (100% training accuracy, 40% test accuracy) into reliable models (85% training, 82% test) that work in the real world.

Understanding these techniques transforms you from someone who fits data to someone who builds models that actually work when deployed.

REFERENCES:

GeeksforGeeks (2025) *Ridge Regression vs Lasso Regression*. https://www.geeksforgeeks.org/machine-learning/ridge-regression-vs-lasso-regression

Kenyansa Felix Amenya (2026). *Ridge vs. Lasso Regression: A Clear Guide to Regularization Techniques*. [online] DEV Community. Available at: https://dev.to/kenyansa/ridge-vs-lasso-regression-a-clear-guide-to-regularization-techniques-14nn.

Medium, M. (2025). *Regularization in Linear Regression: A Deep Dive into Ridge and Lasso*. [online] Medium. Available at: https://medium.com/@maxwienandts/regularization-in-linear-regression-a-deep-dive-into-ridge-and-lasso-3d2853e5e2b0 [Accessed 7 Mar. 2026].

www.datacamp.com. (2022). *Lasso and Ridge Regression Tutorial*. [online] Available at: https://www.datacamp.com/tutorial/tutorial-lasso-ridge-regression.